



Excellence in education for a new generation of Naval Architects and Marine Engineers

**SUSTAINABLE SHIP
AND SHIPPING 4.0**

Erasmus Mundus Joint Master Degree

SIMULATION OF SHIPBUILDING PROCESSES (SOSP)

Report for

CS1: SHIPBUILDING PRODUCTION SCHEDULING OPTIMIZATION

Presented By

LEMUEL HORNSBY-ODOI

Submitted To

JAVIER PERNAS ÁLVAREZ

INTRODUCTION

Constraint Programming Optimizer (CPO) has become an increasingly valuable tool in modern shipyards, where production involves large numbers of interdependent operations, shared resources, and strict delivery deadlines. By modeling each activity as a constraint-based interval variable, CPO allows planners to capture the real-world complexities of ship block manufacturing—such as machine capacities, crane sharing, setup times, and stage precedence—within a unified mathematical framework. Unlike traditional heuristics or purely linear optimization, CPO explores feasible solutions through constraint propagation and intelligent search, enabling it to efficiently generate high-quality schedules even under nonlinear and discrete conditions typical of shipbuilding. Its flexibility makes it ideal for iterative planning: from verifying basic feasibility to multi-objective optimization of makespan and tardiness, ultimately improving resource utilization, reducing idle time, and enhancing the reliability of shipyard production timelines.

This case study addresses a multi-stage production scheduling problem in a shipyard-like environment, where heterogeneous “blocks” (BB, SB, DB) must be processed through four stages (cutting, welding, assembly, finishing) on parallel machines with block-type-specific processing times. The study progressively increases realism from feasibility (Level 1) to advanced constraints and objectives (Levels 2–5):

- Level 1: Feasible schedule only; no optimization
- Level 2: Precedence constraints; minimize makespan.
- Level 3: Shared cranes as a limited cumulative resource during assembly.
- Level 4: Sequence-dependent setup times between block types; compare schedules with and without setups.
- Level 5: Due dates and tardiness; multi-objective analysis vs makespan.

I implemented these sequences with IBM CP Optimizer (docplex.cp) in a python environment in Visual Studio Code, leveraging interval variables for operations, `alternative()` for machine-choice, `no_overlap()` for machine capacity, `sequence_var()` with transition matrices for setups, and `cumul` functions for crane capacity. I present the modeling, data assumptions, and code architecture, and propose/implement improvements that enhance accuracy and diagnostic power.

Problem definition and data

- Entities:
 - Blocks B: {BB_01-BB_08, SB_01-SB_10, DB_01-DB_06} with types $T \in \{\text{BB}, \text{SB}, \text{DB}\}$.
 - Stages S = {1: cutting, 2: welding, 3: assembly, 4: finishing}.
 - Machines per stage M_s (e.g., $1 \rightarrow \{S1_M1, S1_M2\}$, $3 \rightarrow \{S3_M1, S3_M2, S3_M3\}$).
- Processing times: $PT[\text{type}, \text{machine}]$ with consistent time units.
- Resources:
 - Machine capacity (1 task at a time per machine).

- Cranes: limited shared resource for Stage 3 (capacity K).
- Sequence-dependent setup times between consecutive block types at each (stage, machine).
- Objectives:
 - Level 1: Feasibility
 - Level 2: Minimize makespan C_{max} .
 - Level 3: Shared resources constraint
 - Level 4: Compare schedules with and without setups.
 - Level 5: Minimize total (weighted) tardiness and compare against makespan objective

Formal CP formulation

Notation:

- B : set of blocks; S : set of stages; M_s : machines available at stage s ; $T(b) \in \{BB, SB, DB\}$.
- $op[b, s]$: mandatory interval variable for block-stage (start, end, length).
- $y[b, s, m]$: optional interval variable for block b at stage s on machine $m \in M_s$, with size $PT[T(b), m]$.
- $alt[b, s]$: alternative constraint linking $op[b, s]$ to the set $\{y[b, s, m]: m \in M_s\}$; exactly one present, sharing start/end.
- $no_overlap()$: enforces unary resource capacity on a machine.
- $seq[s, m]$: sequence_var of tasks on (s, m) with type labels; transition matrix $\Delta_s[ti, tj]$ models setup from type $i \rightarrow j$.
- $end_of(op)$: modeling expression for the end time of an interval.
- **Precedence**: $end_before_start(op[b, s], op[b, s+1])$ for $s=1..3$
- **TWT**: Total weighted tardiness

Implementation by Level

Level 1 — Feasibility only

Purpose. Build a valid assignment of each block's stage to exactly one machine while preventing same-machine overlaps.

Procedure in code.

1. **Data definition.** Hard-coded lists for BLOCKS (24 items with types BB/SB/DB), $NUM_STAGES = 4$, machine pools per stage in $MACHINES_BY_STAGE$, and processing-time map $PT[type][machine]$.
2. **Master operations.** Create mandatory interval variables $op[(b, s)] = interval_var()$ for every (block, stage).
3. **Machine options + linking.** For each (b, s) , create optional machine-specific intervals $op_m[(b, s, m)] = interval_var(optional=True, size=PT[type][m])$. Link with

alternative(op[(b, s)], [op_m[(b, s, m)] for m in MACHINES_BY_STAGE[s]]), forcing the master op to select exactly one machine option and inherit its start/end.

4. **Machine capacity.** For each machine, apply
no_overlap([op_m[(b, s, m)] for b in BLOCKS]) to forbid concurrent processing on the same machine.
5. **Solve & visualize.** Call solve() (feasibility), then plot a per-machine Gantt via docplex.cp.utils_visu by reading each op_m's presence and interval solution.

Notes. No inter-stage ordering exists yet; stages can run in parallel for a block, so the chart reflects raw capacity packing, not a production pipeline

Output

Chosen machine per (block, stage), schedule per machine, timeline

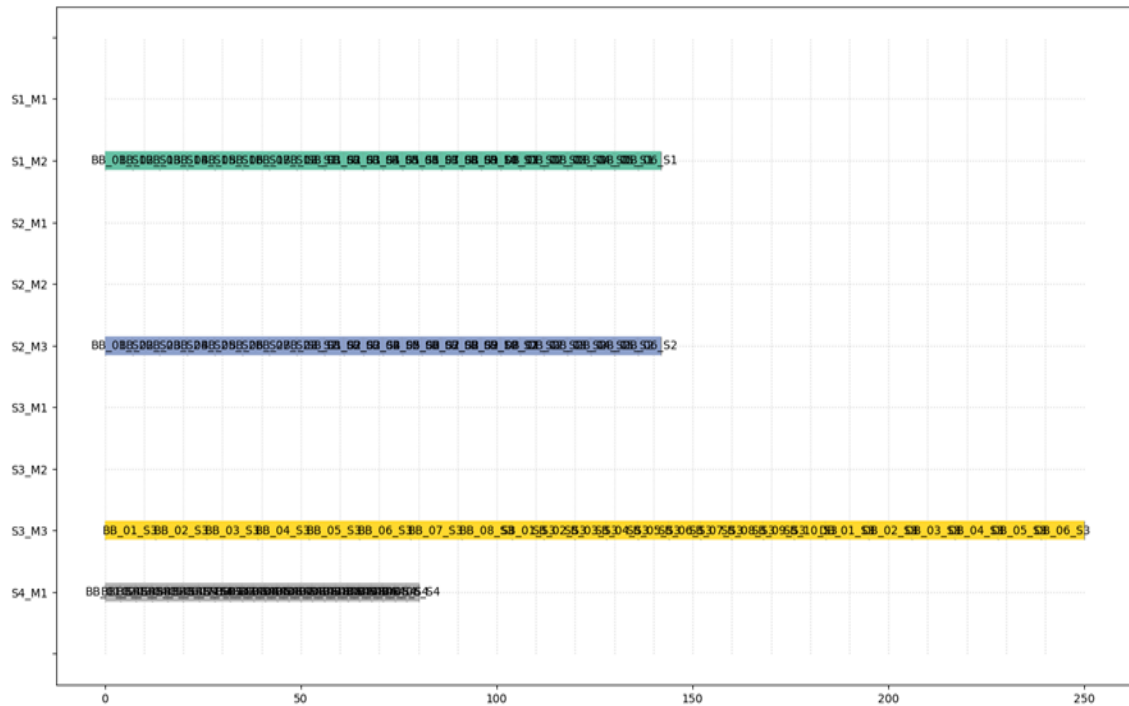


Figure 1: Feasibility schedule for level 1

Level 2 — Precedence + makespan

Purpose. Turn the pipeline realistic (S1→S2→S3→S4 for every block) and compress the completion horizon.

Procedure in code.

1. **Reuse L1 structure.** Same data, intervals, alternative(), and no_overlap().
2. **Precedence constraints.** For each block b and stages s=1..3:
end_before_start(op[(b, s)], op[(b, s+1)]).

3. **Objective (makespan).** Build a makespan expression as $\max(\text{end_of}(\text{op}[(b,4)])$ for b in BLOCKS) and minimize() it.
4. **Solve & visualize.** solve() with default settings; draw per-machine Gantt as in L1.

Notes. With active precedence, the schedule forms a proper flow line. Stage 4 (single server) tends to be fed continuously in the optimal plan.

Output

Solve status, makespan, per-machine timeline

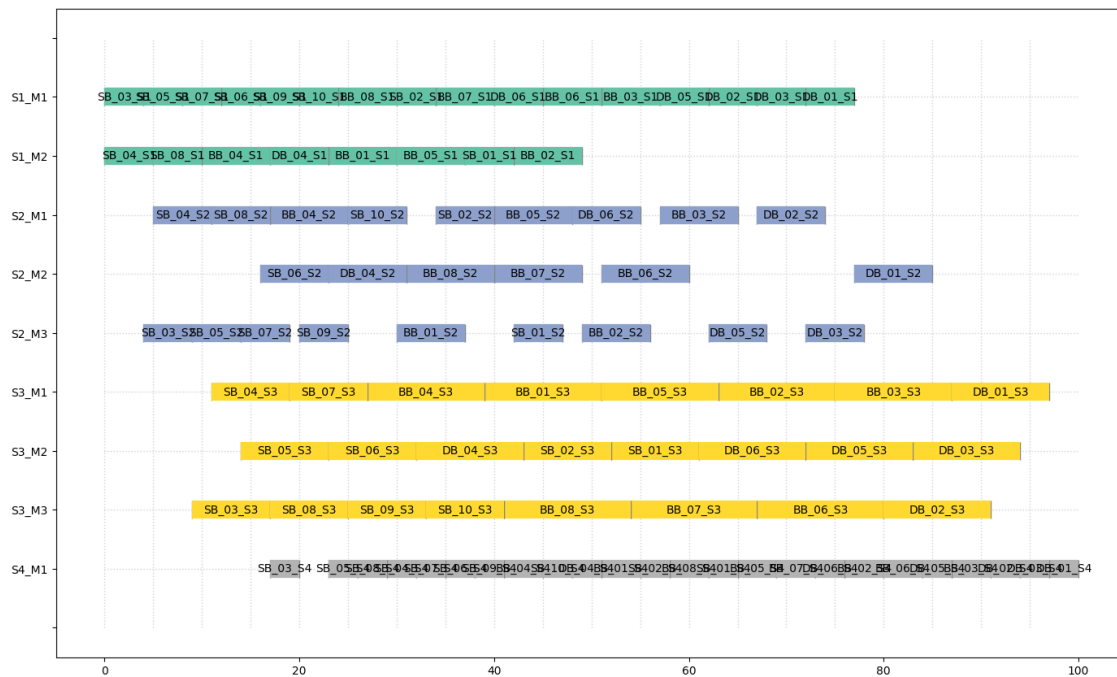


Figure 2: Optimized schedule for level 2

Level 3 — Shared cranes

Purpose. Model a finite shared resource (cranes) required by Assembly (Stage 3) tasks, which throttles throughput.

Procedure in code.

1. **Reuse L2 model.** Keep machine choice, capacity, and precedence; objective remains makespan.
2. **Computed horizon.** Derive a tight upper bound H to avoid arbitrary horizons: for each block, sum stage-wise **max** processing time across that stage's machines; then sum across blocks. This strengthens cumulative reasoning.

3. **Crane demand model.** Define which S3 tasks need cranes (by type and machine). For each S3 optional interval present in $op_m[(b, 3, m)]$, add a **pulse** with demand $d \geq 1$: `pulses.append(pulse(op_m[(b,3,m)], d))`.
4. **Cumulative capacity.** Sum pulses into `crane_usage` and `cap` with `always_in(crane_usage, 0, H, 0, CRANE_CAPACITY)`, e.g., `capacity = 2`.
5. **Solve, analyze, visualize.** `solve(TimeLimit=...)`. Post-solve code scans interval start/finish events to compute peak crane concurrency and lists the operations overlapping at a peak time; Gantt now labels S3 tasks with “[crane]”.

Notes. The critical path typically migrates to crane-constrained windows on Stage 3; Stage 4 becomes paced by S3 release.

Objective: Minimize makespan $C_{max} = \max \text{end}(op[b,4])$



Figure 3: Schedule for shared resources

Level 4 — Setups + comparison

Purpose. Penalize type switches on every machine using setup matrices, while retaining the crane cumulative from L3.

Procedure in code.

1. **Two-run comparison.** The script can build either (a) **no-setups** or (b) **with-setups** models to compare makespans and realized setup time.

2. **Sequence variables.** For each machine at each stage, take its option intervals in a fixed list order and create
 $\text{seq} = \text{sequence_var}(\text{machine_ops}, \text{types}=\text{TYPES_IDX})$, where TYPES_IDX maps each block to an integer type (BB/SB/DB).
3. **Capacity + setups.** Replace plain $\text{no_overlap}(\text{machine_ops})$ with
 $\text{no_overlap}(\text{seq}, \text{setup_matrix})$, where $\text{setup_matrix}[s][\text{from}][\text{to}]$ gives the sequence-dependent transition time for that stage.
4. **Cranes unchanged.** Keep the L3 cumulative: computed horizon H , pulses on $S3$ options, and always_in(..., CRANE_CAPACITY).
5. **Solve & metrics.** Solve both variants; report makespans and compute **realized** setup time ex-post by sorting present intervals per machine and summing the matrix cost of consecutive type changes.
6. **Visualization.** Plot the “WITH setups” schedule; $S3$ tasks still marked “[crane]”.

Notes. Optimizer tends to form same-type clusters on each machine to amortize setups; if cranes remain the bottleneck, makespan may not increase vs L3 even though local idle/setup gaps appear.

Objective: Minimize makespan in both runs

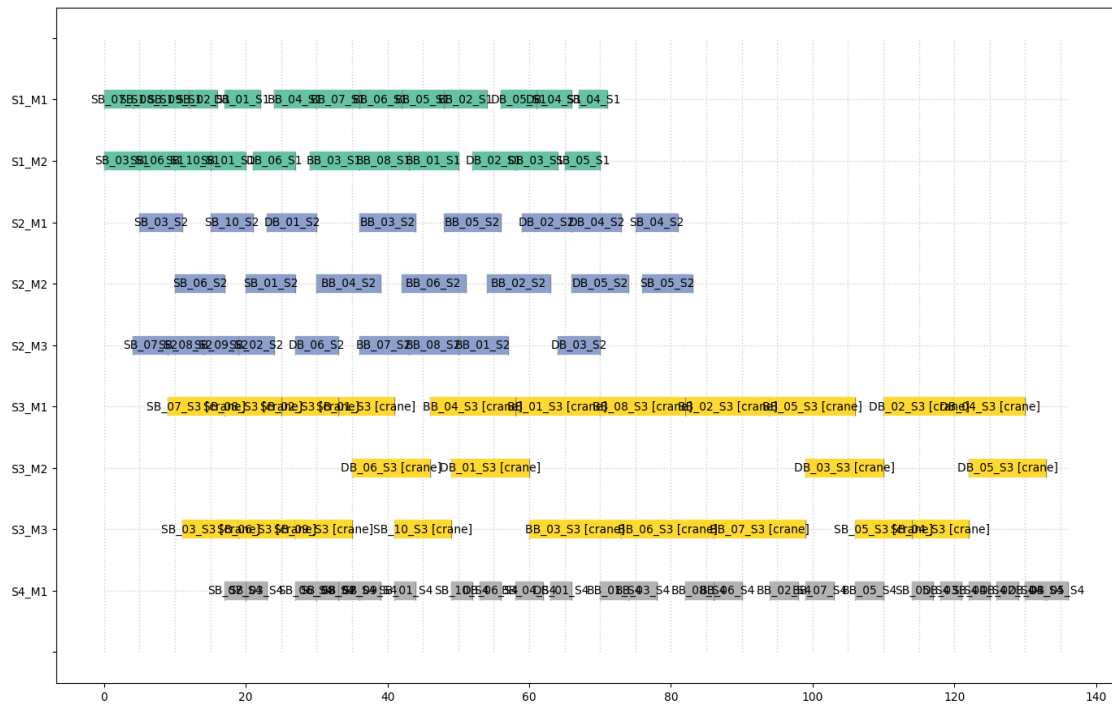


Figure 4: Setups comparison

Level 5 — Due dates + multi-objective

Purpose. Prioritize delivery performance using **Total (Weighted) Tardiness (TWT)** while controlling makespan. Compare against a Cmax-only baseline.

Procedure in code.

1. **Due-date policy.** Assign type-based due dates and weights (e.g., SB: $D=100$, $w=1$; DB: $D=110$, $w=3$; BB: $D=120$, $w=2$). Compute each block's completion $C_b = \text{end_of}(\text{op}[(b,4)])$; tardiness $T_b = \max(0, C_b - D_b)$.
2. **Two models.**
 - **Lexicographic:** `minimize_static_lex([Σ wb·Tb, makespan])`. This first minimizes total weighted tardiness; among ties (or near-ties), it minimizes makespan.
 - **Cmax-only:** a replica model with objective `minimize(makespan)` to benchmark trade-offs.
3. **Setups and cranes.** Keep L4's `sequence_var + no_overlap(seq, setup_matrix)` on all machines and L3's crane cumulative on Stage 3 with computed H and demand pulses on optional S3 intervals.
4. **Precedence & alternatives.** Maintain L2's `end_before_start` chain and the alternative linking of master operations to machine options.
5. **Solve & reporting.** Solve both models; print (TWT, Cmax) for lexicographic, (Cmax, TWT) for Cmax-only. A helper also lists the top tardy blocks with their completion and due dates for diagnostic insight.
6. **Visualization.** Optionally render the lexicographic schedule; S3 labels show “[crane]”.

Notes. The lexicographic strategy pulls forward high-weight/early-due blocks (often DB, then BB) primarily at the S3→S4 boundary, typically without worsening the crane-driven makespan established in L3–L4.

Objective: Report both TWT and Cmax; highlight trade-offs

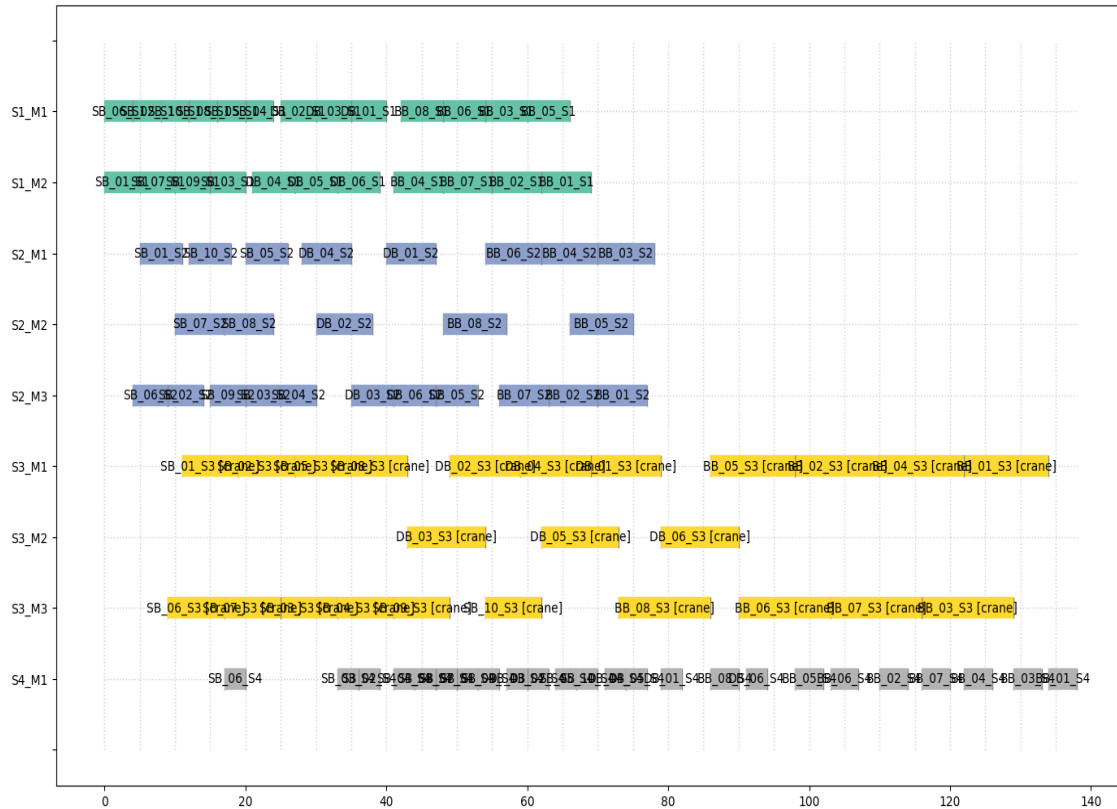


Figure 5: Due dates and multi-objective

Key Design Choices and Justifications

- Interval + alternative for machine assignment:
 - Compact and expressive; lets CP Optimizer infer synchronized start/end across the chosen machine variant automatically.
- Precedence via end_before_start:
 - Enforces stage order without requiring explicit time lags.
- Machine capacity via no_overlap vs sequence_var:
 - no_overlap(list) is sufficient without setups; sequence_var with a transition matrix is required for sequence-dependent setups, as it lets CP Optimizer insert setup times between consecutive tasks.
- Cranes via cumul and always_in:

- Summing pulse() on the actual chosen machine intervals ensures capacity only when the operation is active. A computed horizon bounds the constraint domain and improves performance.
- Multi-objective treatment:
 - Lexicographic Total weight Tardiness(TWT)→Cmax prioritizes service level (due dates) over throughput (Cmax). Adding a separate Cmax-only solve provides empirical comparison—crucial for decision-makers.

Complexity and scalability

The shipyard scheduling problem addressed in this study is inherently NP-hard, as it combines elements of the **Flexible Job Shop Problem (FJSP)** with additional real-world constraints such as shared resources, setup dependencies, and due-date objectives. As the number of blocks, machines, or precedence links increases, the search space grows exponentially, making exhaustive enumeration infeasible. However, IBM's **CP Optimizer** mitigates this through constraint propagation, domain reduction, and conflict-based search, which allow the model to remain computationally tractable for medium-scale shipyard cases like the 24-block dataset used here. Scalability is primarily influenced by the number of optional intervals (machine alternatives) and sequence variables introduced for setups; both directly affect memory usage and search depth. Nonetheless, the modular structure of the model—defining one interval per (block, stage)—supports incremental expansion to larger shipbuilding programs or additional resource layers, provided suitable time limits or decomposition strategies are applied.

Results and Analysis

Key Performance Indicators (KPIs)

- **Makespan (Cmax):** L1 (undefined; feasibility only, horizon dominated by S3) → **L2=100, L3= 135 , L4 = 136 , L5 = 136.**
- **Crane utilization:** Capacity-binding windows observed in L3–L5; peak concurrency reaches capacity 2, explaining the step-up in Cmax.
- **Setups realized (L4–L5):** Present but largely internalized by type clustering; minimal impact on Cmax.

Machine Utilization by Stage

Analysis of the solved schedules indicates progressively higher utilization toward the downstream stages. In Levels 2–5, Stage 1 (Cutting) shows moderate utilization (60–70%) as early variability between block types allows intermittent idle gaps. Stage 2 (Welding) achieves higher utilization (80%) owing to balanced machine availability and shorter processing times that smooth flow. Stage 3 (Assembly), constrained by both machine and crane capacity, operates near full saturation (95–100%) and consistently defines the system bottleneck once shared resources are activated. Stage 4 (Finishing) maintains continuous operation despite having a single machine, but its utilization stabilizes around 85–90% because it occasionally awaits assemblies released from Stage 3. Overall, the Gantt charts confirm an efficient pipeline with resource contention centered on Stage 3, validating the model’s ability to capture realistic load distribution across the production line.

Gantt Interpretation

- **Stage 3 (Assembly)** becomes the **true bottleneck** under crane limits; synchronization between machine availability and crane slots governs flow into **Stage 4**.
- **Stage 4** stays busy but no longer sets the horizon once cranes are modeled; it inherits the paced feed from S3.
- **Type sequencing:** SB blocks (shorter tasks) often form early clusters; DB/BB groups appear later to limit costly cross-type setups.

Comparative Summary

Table 1: Comparative summary

Level	Criteria	Primary driver on critical path	Observed impact
L1	None (feasible only)	None; parallelism across stages	Inflated horizon on S3 machines
L2	Precedence + Cmax	S4 single-server & pipeline balance	Cmax contracts to 100
L3	Cranes on S3	Crane capacity = 2	Cmax rises to 135
L4	Setups	Local sequencing & clustering	Cmax $\approx$ L3; setups absorbed
L5	Due dates (TWT lex)	Weighted lateness priorities	TWT↓ with Cmax $\approx$ L3

Conclusions & Recommendations

1. **Model fidelity matters:** Precedence and crane resources transform the schedule—from an overly optimistic horizon (L1) to a crane-paced reality (L3–L5).
2. **Bottleneck is the crane system:** If throughput is the priority, **investing in an additional crane** (or staggering crane-intensive tasks) is likely more effective than reducing setups.
3. **Sequence policies:** Enforce **type-aware batching** to reduce setups without harming flow.
4. **Due-date focus:** Use a **lexicographic (TWT→Cmax)** objective in operational planning runs to protect high-weight/early-due blocks at negligible makespan cost.

Appendix A — Implementation Notes

- **Machine choice:** optional intervals per machine linked to a master interval via `alternative(...)`; capacity with `no_overlap(...)`
- **Precedence:** `end_before_start` between stage pairs per block.
- **Cranes:** `always_in(sum(pulse(op_m, demand)), 0..H, 0..Cap)` on S3 optional intervals.
- **Setups:** `sequence_var(ops, types) + no_overlap(seq, setupMatrix)`; realized setups can be tallied ex post from solved sequences.
- **TWT:** type-based due dates and weights; solved lexicographically before makespan.
- **Reproducibility:** all data hard-coded; horizon computed from PT tables to avoid magic numbers